

WEEK 4 ACTIVITY

IMPLEMENT CRUD FUNCTIONALITY IN YOUR CURRENT PROJECT



Sawatâ: A Smart Gambling Blocker for Recovery

Advanced Mobile Programming

MAIN CRUD ENTITY
Journal Entries

DATABASE
Firebase Cloud Firestore

Prepared August 17, 2026

Module documented: lib/screens/user/journal — lib/services/user_data_service.dart

Overview

CRUD functionality was implemented in the **Journal** module of Sawatâ, the app's private space for a user to log their mood and reflections during recovery. Every journal entry is stored per-user in **Firestore**, under the path `users/{uid}/journal_entries/{entryId}`, and the Journal screen reads that collection through a live Firestore snapshot listener, so any create, update, or delete is reflected in the interface immediately — no manual refresh required. Access is scoped to the signed-in user only, both in the query itself and in the deployed Firestore security rules.

Operation	Implementation
Create	Create and save a new journal entry
Read	Retrieve and display saved journal entries
Update	Edit an existing journal entry
Delete	Delete an existing journal entry
Validation	Validate journal input before saving
Storage	Firestore

The sections that follow walk through each operation as it is actually implemented in the codebase, referencing the real widgets, methods, and Firestore fields involved.

Create Operation

A new entry is created from the Journal screen's **New Entry** button, which calls `JournalScreen._addEntry()`. This opens a full-screen composer (`showAddJournalSheet`) with three fields:

- A **mood** selector — one of four fixed options: Great, Good, Okay, or Stressed.
- An optional **title**, capped at 80 characters with a live counter.
- A required **body** (the entry content), capped at 2,000 characters with a live counter.

The **Save Entry** button stays disabled until the body field contains non-empty text. On save, the composer returns the entered mood, title, and body to the Journal screen, which calls `UserDataService.addJournalEntry(uid, mood:, title:, body:)`. That method writes a new document to `users/{uid}/journal_entries` with an auto-generated Firestore document ID and four fields: `date` (the save-time timestamp), `mood`, `title`, and `body`. A “Entry saved” confirmation snackbar appears, and the new entry shows up in the list immediately through the screen's live Firestore stream.

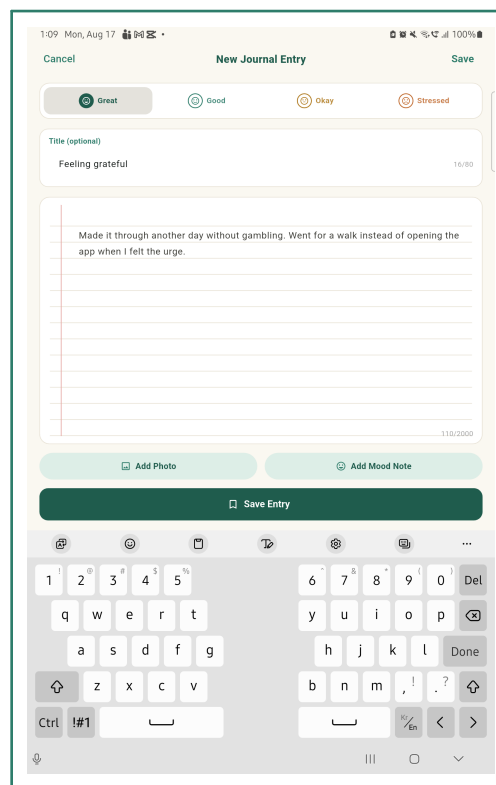


Figure 1. Create Operation — The user creates a new journal entry by selecting a mood and entering their journal content before saving it to Firestore.

Read Operation

The Journal screen displays entries through a `StreamBuilder` listening to `UserDataService.watchJournalEntries(uid)`. That method queries `users/{uid}/journal_entries`, ordered by `date` descending, and maps each document snapshot back into a `JournalEntry` object. Because the query is built from the currently authenticated user's own UID (`FirebaseAuth.instance.currentUser?.uid`), a user only ever retrieves their own entries.

On screen, the list is grouped into **Today**, **This Week**, and **Earlier** sections based on each entry's date relative to the current date. Each entry renders as a card showing the date, time, mood icon and label, and a two-line preview of the entry body; tapping a card opens a detail dialog with the full title and body. If the collection is empty, the screen shows an empty state (“No journal entries yet...”) with a “Write an entry” call-to-action instead of a blank list.

Because this is a live snapshot stream rather than a one-time fetch, any entry added, edited, or deleted — from this device or another session signed into the same account — updates the list in real time.

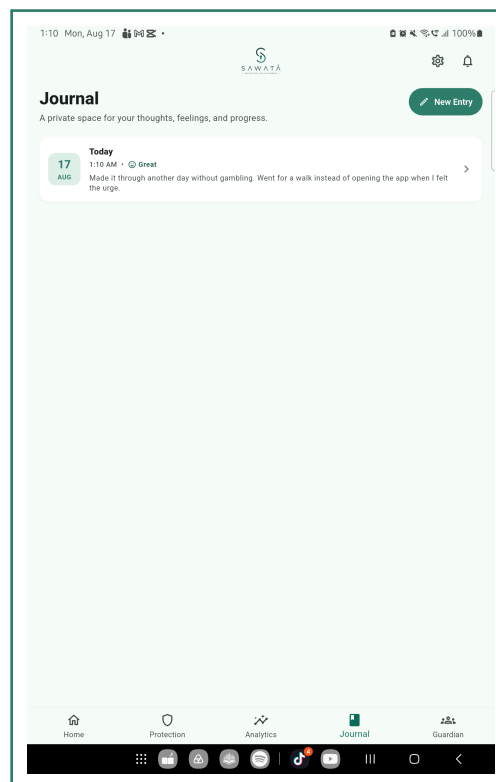


Figure 2. Read Operation — Previously saved journal entries are retrieved from Firebase Cloud Firestore and displayed in the Journal screen.

Update Operation

Tapping a journal entry opens a detail dialog with the entry's full title and body. From there, tapping **Edit** reopens the same composer used for Create (`showAddJournalSheet`), this time pre-filled with the entry's existing mood, title, and body — the two flows share one form so they cannot visually drift apart. The dialog title changes to “Edit Journal Entry” and the save button reads “Save Changes.”

The same body-required validation from Create applies here. On save, the screen calls `UserDataService.updateJournalEntry(uid, entryId, mood:, title:, body:)`, which issues a Firestore `.doc(entryId).update({...})` against that specific document — only the `mood`, `title`, and `body` fields are overwritten; the entry's original `date` field is left untouched, so editing an entry does not change when it was originally written. A “Entry updated” snackbar confirms the save, and the change appears in the list immediately via the live stream.

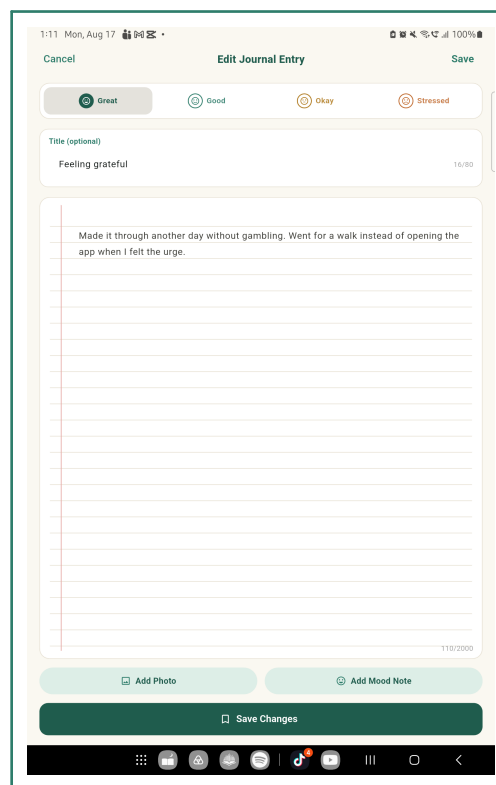


Figure 3. Update Operation — An existing journal entry can be edited and the updated information is saved back to Firestore.

Delete Operation

Each journal entry card is wrapped in a `Dismissible` widget. Swiping a card from right to left reveals a red delete panel underneath as the card slides away; releasing the swipe removes the card and calls `UserDataService.removeJournalEntry(uid, entryId)`, which issues a Firestore `.doc(entryId).delete()` against that document. An “Entry deleted” snackbar confirms the action, and the entry disappears from every screen reading the live stream.

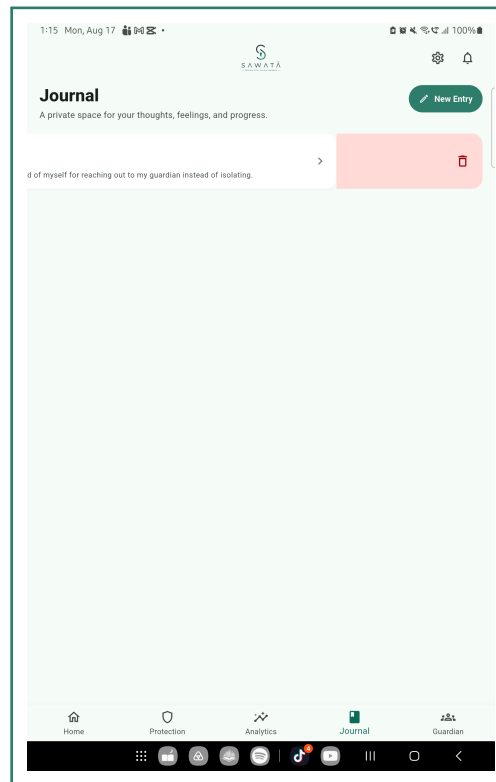


Figure 4. Delete Operation — The user can remove an existing journal entry by swiping it away, which permanently deletes it from Firestore.

No confirmation dialog

Unlike some other destructive actions in the app (for example, cancelling a sent guardian invite, which opens a confirm dialog before writing to Firestore), journal entry deletion currently has **no confirmation step**. The swipe gesture itself — a deliberate drag rather than a single tap — is the only safeguard against an accidental delete. This is stated here as-is rather than assumed to exist.

Data Validation

The following validation is actually enforced in the code, on both Create and Update:

- **Body is required.** The Save button is disabled while the body field is empty or whitespace-only, and the save handler re-checks this before writing to Firestore — an entry with no content can never be saved.
- **Title is optional**, limited to 80 characters, with a live on-screen counter.
- **Body is limited to 2,000 characters**, also with a live counter.
- **Mood is constrained to four fixed options** (Great, Good, Okay, Stressed) chosen from selectable chips rather than typed freely, so no invalid mood value can be entered.

At the database layer, Firestore security rules restrict all read and write access on `users/{uid}/journal_entries` to the authenticated owner (`request.auth.uid == uid`). This is enforced server-side regardless of what the client sends, but it is an ownership check, not a content check — field-level rules such as length limits are enforced only on the client, not independently re-verified by the security rules.

Persistent Data Storage

All journal data is stored in **Firebase Cloud Firestore**, under `users/{uid}/journal_entries/{entryId}` — one document per entry, holding the `date`, `mood`, `title`, and `body` fields. Because Firestore is a cloud database tied to the signed-in account rather than local device storage, entries remain available after the app is closed, the device is restarted, or the user signs in on a different device with the same account. The Journal screen reads this collection through a live snapshot listener, so create, update, and delete operations are all reflected across the interface immediately.

Conclusion

The Journal module of Sawatâ successfully demonstrates complete CRUD functionality by allowing users to create, read, update, and delete journal entries. Firebase Cloud Firestore provides persistent data storage, while validation and confirmation mechanisms help maintain accurate and safe user data.